

ΚΕΦΑΛΑΙΟ 2

Δομές Ελέγχου στη Fortran

Συνθήκες • Λογικές εκφράσεις • IF / ELSE IF / ELSE • Εμφώλευση

Στόχος του κεφαλαίου

Να περάσουμε από τα σειριακά προγράμματα του Κεφαλαίου 1 σε προγράμματα που μπορούν να παίρνουν αποφάσεις: να ελέγχουν μια συνθήκη και να επιλέγουν ποιο τμήμα του κώδικα θα εκτελεστεί.

Βάση του κεφαλαίου

Οι σημειώσεις οργανώθηκαν με βάση το FORTRAN_2025 και τις Ασκήσεις 01, με σύνδεση προς το Lab 5. Η σειρά και η διατύπωση έχουν προσαρμοστεί ώστε να λειτουργούν ως προσωπικό εγχειρίδιο μελέτης.

Τρόπος μελέτης

Διάβασε πρώτα τη λογική, γράψε μόνος σου τα μικρά παραδείγματα και μετά λύσε τις ασκήσεις. Στόχος δεν είναι να απομνημονεύσεις ένα πρότυπο IF, αλλά να μετατρέπεις την εκφώνηση σε περιπτώσεις και συνθήκες.

□ Κεντρική ιδέα

Μια συνθήκη είναι μια ερώτηση που έχει απάντηση TRUE ή FALSE. Το IF χρησιμοποιεί αυτή την απάντηση για να αποφασίσει ποιον δρόμο θα ακολουθήσει το πρόγραμμα.

Περιεχόμενα

- 2.1 Από τη σειριακή εκτέλεση στον έλεγχο ροής
- 2.2 Λογικές τιμές και συνθήκες
- 2.3 Τελεστές σύγκρισης
- 2.4 Λογικοί τελεστές
- 2.5 Η δομή IF ... THEN
- 2.6 Η δομή IF ... THEN ... ELSE
- 2.7 Η δομή ELSE IF
- 2.8 Εμφωλευμένα IF
- 2.9 Έλεγχος εγκυρότητας και διερεύνηση περιπτώσεων
- 2.10 Σύγκριση πραγματικών αριθμών
- 2.11 Από εκφώνηση σε IF
- 2.12 Συχνά λάθη
- 2.13 Ασκήσεις εφαρμογής
- 2.14 Checklist πριν προχωρήσουμε

Σύνδεση με τα επόμενα κεφάλαια

Οι ίδιες λογικές συνθήκες θα εμφανιστούν αργότερα μέσα σε DO WHILE, IF ... EXIT και IF ... CYCLE, όπως φαίνεται στο Lab 5.

2.1 Από τη σειριακή εκτέλεση στον έλεγχο ροής

Στο Κεφάλαιο 1 δουλέψαμε κυρίως με τη λογική δεδομένα → υπολογισμοί → αποτελέσματα. Όταν όμως η εκφώνηση λέει «αν...», «διαφορετικά...» ή «ανάλογα με...», χρειαζόμαστε έλεγχο ροής.

Παράδειγμα: «Διάβασε έναν ακέραιο και έλεγξε αν είναι άρτιος». Η άρτια και η μη άρτια περίπτωση οδηγούν σε διαφορετικό αποτέλεσμα.

Σκέψου πρώτα τον αλγόριθμο

Ποια ερώτηση πρέπει να κάνω στον υπολογιστή; Μετά μετέτρεψε αυτή την ερώτηση σε λογική έκφραση.

Δεδομένο: x

Ερώτηση: Είναι το x μεγαλύτερο από 0;

Αν ΝΑΙ -> κλάδος Α

Αν ΟΧΙ -> κλάδος Β

2.2 Λογικές τιμές και συνθήκες

Η Fortran διαθέτει τον τύπο δεδομένων LOGICAL. Οι βασικές λογικές τιμές είναι .TRUE. και .FALSE..

```
logical :: found, valid
```

```
found = .true.  
valid = .false.
```

Πολύ συχνά η λογική τιμή προκύπτει από σύγκριση:

```
x > 0  
a == b  
n /= 0  
temperature <= 25.5
```

Κάθε τέτοια έκφραση αξιολογείται σε TRUE ή FALSE.

Εκχώρηση ≠ σύγκριση

Γραφή	Τι σημαίνει
x = 5	Εκχώρηση: αποθήκευσε 5 στο x.
x == 5	Σύγκριση: είναι το x ίσο με 5;

□ Κανόνας

Στο IF βάζουμε μια λογική έκφραση μέσα στην παρένθεση.

2.3 Τελεστές σύγκρισης

Οι βασικοί τελεστές σύγκρισης είναι:

Fortran	Σημασία	Παράδειγμα
==	ίσο	x == 10
/=	διάφορο	x /= 0
>	μεγαλύτερο	x > 0
<	μικρότερο	x < 0
>=	μεγαλύτερο ή ίσο	x >= 5
<=	μικρότερο ή ίσο	x <= 5

Αν $x = 10$: $x == 10 \rightarrow \text{TRUE}$, $x > 10 \rightarrow \text{FALSE}$, $x >= 10 \rightarrow \text{TRUE}$, $x /= 10 \rightarrow \text{FALSE}$.

Το υλικό του καθηγητή δίνει και παλαιότερες μορφές .GT., .LT., .EQ., .NE., .GE., .LE. Καλό είναι να τις αναγνωρίζεις, ενώ στις δικές μας σημειώσεις προτιμάμε τις σύγχρονες μορφές.

Συμπλήρωμα 2.3 — Όλοι οι τελεστές σύγκρισης

Κάθε γραμμή είναι ένα ζευγάρι ισοδύναμων τελεστών. Όλοι οι 6 φαίνονται καθαρά:

Σύγχρονη μορφή	Παλαιότερη μορφή
<	.LT.
<=	.LE.
==	.EQ.
/=	.NE.
>=	.GE.
>	.GT.

▢ Παραδείγματα

<code>x < 5</code>	↔	<code>x .LT. 5</code>	→ «το x είναι μικρότερο από 5»
<code>x <= 5</code>	↔	<code>x .LE. 5</code>	→ «το x είναι μικρότερο ή ίσο με 5»
<code>x == 5</code>	↔	<code>x .EQ. 5</code>	→ «το x είναι ίσο με 5»
<code>x /= 5</code>	↔	<code>x .NE. 5</code>	→ «το x είναι διαφορετικό από 5»
<code>x >= 5</code>	↔	<code>x .GE. 5</code>	→ «το x είναι μεγαλύτερο ή ίσο με 5»
<code>x > 5</code>	↔	<code>x .GT. 5</code>	→ «το x είναι μεγαλύτερο από 5»

▢ Τι κρατάμε

Δεν έχουμε 12 διαφορετικούς τελεστές: έχουμε 6 συγκρίσεις με 2 διαφορετικές γραφές.

Στις δικές μας ασκήσεις χρησιμοποιούμε τις σύγχρονες μορφές.

Τις παλιές μορφές τις αναγνωρίζουμε όταν εμφανίζονται στο υλικό του καθηγητή ή σε παλιό κώδικα.

2.4 Λογικοί τελεστές

Το FORTRAN_2025 παρουσιάζει τους .NOT., .AND., .OR., καθώς και .EQV. και .NEQV. Στο βασικό επίπεδο δίνουμε έμφαση στους τρεις πρώτους.

Τελεστής	Ιδέα	Παράδειγμα
.AND.	και οι δύο συνθήκες TRUE	$x > 0$.and. $x < 10$
.OR.	τουλάχιστον μία TRUE	$x < 0$.or. $x > 10$
.NOT.	αντιστροφή	.not. found

Έλεγχος διαστήματος

```
if (x >= 0 .and. x <= 100) then
  print *, "Valid"
end if
```

Η συνθήκη διαβάζεται: «το x είναι ≥ 0 και ταυτόχρονα ≤ 100 ».

Προσοχή στη λογική

Με .AND. πρέπει να είναι TRUE και οι δύο συνθήκες. Με .OR. αρκεί μία.

Συμπλήρωμα 2.4 — Όλοι οι λογικοί τελεστές

Ο πίνακας του καθηγητή περιλαμβάνει και τους 5 λογικούς τελεστές:

Τελεστής	Σημασία
.NOT.	άρνηση
.AND.	σύζευξη (και)
.OR.	διάζευξη (ή)
.EQV.	ισοδυναμία
.NEQV.	ανισοδυναμία

▢ Παραδείγματα

.NOT. a → αν a=TRUE, αποτέλεσμα FALSE
a .AND. b → TRUE μόνο όταν a=TRUE και b=TRUE
a .OR. b → TRUE όταν τουλάχιστον ένα από a,b είναι TRUE
a .EQV. b → TRUE όταν a και b έχουν την ίδια λογική τιμή
a .NEQV. b → TRUE όταν a και b έχουν διαφορετική λογική τιμή

▢ Τι κρατάμε

.AND. = «και», .OR. = «ή», .NOT. = «αντίθετο/άρνηση».
.EQV. και .NEQV. δεν είναι «παλιές μορφές»· είναι δύο επιπλέον λογικοί τελεστές.
Τους κρατάμε ώστε να μπορούμε να διαβάσουμε σωστά όλο το υλικό του καθηγητή.

2.5 Η δομή IF ... THEN

Η απλούστερη μορφή είναι:

```
if (συνθήκη) then  
    προτάσεις  
end if
```

Αν η συνθήκη είναι TRUE, εκτελούνται οι προτάσεις ανάμεσα στο THEN και το END IF. Αν είναι FALSE, το πρόγραμμα τις προσπερνά.

Παράδειγμα

```
program positive  
    implicit none  
    integer :: x  
    read *, x  
    if (x > 0) then  
        print *, "The number is positive"  
    end if  
end program positive
```

Μικρός έλεγχος

Αν $x = -4$, η συνθήκη $x > 0$ είναι FALSE και το σώμα του IF δεν εκτελείται.

2.6 Η δομή IF ... THEN ... ELSE

Όταν έχουμε δύο περιπτώσεις:

```
if (συνθήκη) then
    προτάσεις_A
else
    προτάσεις_B
end if
```

Ακριβώς ένας από τους δύο κλάδους θα εκτελεστεί.

Παράδειγμα — άρτιος / περιττός

```
program parity
    implicit none
    integer :: n
    read *, n
    if (mod(n, 2) == 0) then
        print *, "Even"
    else
        print *, "Odd"
    end if
end program parity
```

Η αντίστοιχη άσκηση υπάρχει ρητά στις Ασκήσεις 01 και χρησιμοποιεί $\text{MOD}(n,2)=0$ για άρτιους.

Σύνδεση με Κεφάλαιο 1

Το MOD το μάθαμε ήδη. Τώρα το συνδυάζουμε με σύγκριση και IF.

2.7 Η δομή ELSE IF

Για περισσότερες από δύο περιπτώσεις χρησιμοποιούμε αλυσίδα ELSE IF. Ο πρώτος κλάδος του οποίου η συνθήκη είναι TRUE εκτελείται και οι υπόλοιποι αγνοούνται.

```
if (condition1) then
  ...
else if (condition2) then
  ...
else if (condition3) then
  ...
else
  ...
end if
```

Παράδειγμα — κατηγοριοποίηση βαθμού

```
if (grade > 10.0) then
  print *, "Invalid grade"
else if (grade >= 9.0) then
  print *, "A"
else if (grade >= 8.0) then
  print *, "B"
else if (grade >= 7.0) then
  print *, "C"
else if (grade >= 6.0) then
  print *, "D"
else if (grade >= 5.0) then
  print *, "E"
else if (grade >= 0.0) then
  print *, "F"
else
  print *, "Invalid grade"
end if
```

□ Κανόνες σειράς

Η σειρά των συνθηκών έχει σημασία. Μια γενική συνθήκη μπορεί να «πιάσει» μια τιμή πριν προλάβει να εξεταστεί μια ειδικότερη.

2.8 Εμφωλευμένα IF

Όταν η δεύτερη απόφαση έχει νόημα μόνο αφού πάρουμε την πρώτη, έχουμε εμφωλευμένο IF.

```
if (a == 0) then
  ...
else
  if (d >= 0) then
    ...
  else
    ...
  end if
end if
```

Στο FORTRAN_2025 η εμφώλευση χρησιμοποιείται, μεταξύ άλλων, για τη διερεύνηση του τριωνύμου: πρώτα ελέγχεται αν $a=0$ και στη συνέχεια, όταν χρειάζεται, το πρόσημο της διακρίνουσας.

Εμφώλευση = πλήρης δομή μέσα σε έναν κλάδο

Το εσωτερικό IF πρέπει να βρίσκεται ολόκληρο μέσα σε έναν κλάδο του εξωτερικού IF, από το δικό του IF μέχρι το δικό του END IF.

Μικρό παράδειγμα

```
if (x >= 0) then
  if (x == 0) then
    print *, "Zero"
  else
    print *, "Positive"
  end if
else
  print *, "Negative"
end if
```

Το IF χρησιμοποιείται για έλεγχο εγκυρότητας αλλά και για πλήρη διερεύνηση ενός προβλήματος.

Παράδειγμα — ακτίνα σφαίρας

```
read *, r

if (r < 0.0) then
  print *, "Invalid radius"
else
  volume = 4.0/3.0 * pi * r**3
  area   = 4.0 * pi * r**2
  print *, volume, area
end if
```

Η αντίστοιχη άσκηση των Ασκήσεων 01 ζητά μήνυμα λάθους για αρνητική ακτίνα.

Παράδειγμα — $ax = b$

Η εκφώνηση απαιτεί τρεις περιπτώσεις: $a \neq 0 \rightarrow$ μία λύση, $a=0$ και $b=0 \rightarrow$ άπειρες λύσεις, $a=0$ και $b \neq 0 \rightarrow$ καμία λύση.

```
if (a /= 0.0) then
  x = b / a
  print *, "One solution:", x
else
  if (b == 0.0) then
    print *, "Infinitely many solutions"
  else
    print *, "No solution"
  end if
end if
```

Εδώ το σημαντικό είναι η διερεύνηση, όχι η αποστήθιση του κώδικα.

2.10

Σύγκριση πραγματικών αριθμών

Πριν δούμε το παράδειγμα του καθηγητή, χρειαζόμαστε δύο μικρές έννοιες που εμφανίζονται για πρώτη φορά εδώ: τη γραφή με εκθέτη και τη συνάρτηση ABS.

2.10.1 — Τι σημαίνει το 1.e-4;

Η Fortran μπορεί να γράψει έναν πραγματικό αριθμό με εκθέτη. Το e σημαίνει ότι ο αριθμός πολλαπλασιάζεται με δύναμη του 10.

$$1.e-4 = 1 \times 10^{-4} = 0.0001$$

Πώς διαβάζεται;

e-4 σημαίνει «επί 10 στη δύναμη -4». Ο αρνητικός εκθέτης δίνει έναν πολύ μικρό αριθμό. Άρα το 1.e-4 είναι απλώς άλλος τρόπος να γράψουμε το 0.0001.

$$1.e-2 = 0.01 \quad 1.e-3 = 0.001 \quad 1.e-4 = 0.0001$$

Άρα, αν δεις σε κώδικα έναν αριθμό της μορφής 1.e-4, δεν είναι νέος τελεστής ή συνάρτηση· είναι απλώς ένας πραγματικός αριθμός γραμμένος με εκθέτη.

2.10.2 — Τι είναι το ABS;

Το ABS είναι συνάρτηση που δίνει την απόλυτη τιμή ενός αριθμού, δηλαδή την τιμή χωρίς το πρόσημό του.

$$\begin{aligned} \text{abs}(5) &= 5 \\ \text{abs}(-5) &= 5 \\ \text{abs}(0) &= 0 \end{aligned}$$

Έτσι, η έκφραση $\text{abs}(a-b)$ μας λέει πόσο απέχουν αριθμητικά οι δύο τιμές, χωρίς να μας ενδιαφέρει ποια είναι μεγαλύτερη.

Παράδειγμα

$$a = 10 \text{ και } b = 10.00001 \rightarrow a-b = -0.00001 \rightarrow \text{abs}(a-b) = 0.00001.$$

2.10.3 — Τι σημαίνει «ανοχή»;

Όταν δύο REAL αριθμοί προκύπτουν από υπολογισμούς, μπορεί ο υπολογιστής να αποθηκεύσει μια τιμή ελάχιστα διαφορετική από αυτή που περιμένουμε μαθηματικά. Γι' αυτό, σε τέτοιες συγκρίσεις, αντί να ζητήσουμε «είναι ακριβώς ίσα;», μπορούμε να ζητήσουμε «είναι αρκετά κοντά;».

Η μικρή επιτρεπόμενη διαφορά λέγεται ανοχή. Για παράδειγμα, αν επιλέξουμε ανοχή $\text{tol} = 0.0001$, θεωρούμε δύο αριθμούς αρκετά κοντά όταν η απόλυτη διαφορά τους είναι μικρότερη από 0.0001.

```
if (abs(a-b) < tol) then
```

Διαβάζεται: «αν η απόλυτη διαφορά των a και b είναι μικρότερη από την ανοχή tol , τότε θεωρούμε τις δύο τιμές αρκετά κοντά».

2.10.4 — Το παράδειγμα του καθηγητή

Στο FORTRAN_2025 εμφανίζεται το παράδειγμα:

```
a = 1000.0 / 30.0  
a = a * 30.0
```

Μαθηματικά θα περιμέναμε να επιστρέψουμε ακριβώς στο 1000. Όμως οι πραγματικοί αριθμοί έχουν πεπερασμένη ακρίβεια αναπαράστασης, οπότε ένα αποτέλεσμα υπολογισμού μπορεί να είναι ελάχιστα διαφορετικό. Το υλικό του καθηγητή προτείνει τότε έλεγχο της μορφής:

```
if (abs(a-b) < 1.e-4) then  
print *, "Equal"  
end if
```

Τώρα μπορούμε να διαβάσουμε όλη την πρόταση

$\text{abs}(a-b)$ = απόλυτη διαφορά, $1.e-4 = 0.0001$, άρα ο έλεγχος λέει: «η διαφορά είναι μικρότερη από 0.0001;». Αν ναι, θεωρούμε τις τιμές αρκετά κοντά για αυτή την εφαρμογή.

□ Για τις δικές μας ασκήσεις

Δεν χρειάζεται να χρησιμοποιούμε ανοχή σε κάθε σύγκριση. Σε απλούς ελέγχους ορίων, όπως $x \geq \theta$ ή $x \leq 10$, χρησιμοποιούμε κανονικά τους τελεστές σύγκρισης που μάθαμε. Η ανοχή αφορά κυρίως τη σύγκριση αποτελεσμάτων υπολογισμών REAL.

2.10 Σύγκριση πραγματικών αριθμών

Το FORTRAN_2025 δείχνει ότι η απόλυτη ισότητα δύο REAL που προέκυψαν από υπολογισμούς μπορεί να είναι προβληματική λόγω σφαλμάτων αναπαράστασης.

```
a = 1000.0 / 30.0  
a = a * 30.0
```

```
if (a == 1000.0) then  
    print *, "Equal"  
end if
```

Το ίδιο υλικό δείχνει ως ασφαλέστερη λογική έναν έλεγχο με ανοχή:

```
if (abs(a - b) < 1.e-4) then  
    ...  
end if
```

Για τώρα

Σε απλές συγκρίσεις ορίων χρησιμοποίησε κανονικά $>$, $<$, $>=$, $<=$. Η ανοχή είναι ιδιαίτερα σημαντική όταν συγκρίνουμε αποτελέσματα υπολογισμών.

2.11 Από εκφώνηση σε IF

Χρησιμοποιούμε τη μεθοδολογία του Κεφαλαίου 1 και προσθέτουμε ένα νέο βήμα: εντοπίζουμε τις περιπτώσεις.

Βήμα	Τι κάνω	Ερώτηση
1	Δεδομένα	Τι δίνει ο χρήστης;
2	Ζητούμενα	Τι πρέπει να υπολογίσουμε/εκτυπώσουμε;
3	Περιπτώσεις	Πότε αλλάζει ο τρόπος υπολογισμού;
4	Συνθήκες	Ποια ερώτηση περιγράφει κάθε περίπτωση;
5	IF δομή	IF, ELSE IF, ELSE ή εμφώλευση;
6	Έλεγχος	Καλύπτονται όλες οι δυνατές εισοδοι;

Παράδειγμα σκέψης

«Διάβασε έναν αριθμό. Αν είναι αρνητικός, εμφάνισε μήνυμα. Διαφορετικά υπολόγισε ...»

Δεδομένο: x

Ειδική περίπτωση: $x < 0$

Κλάδος TRUE: μήνυμα

Κλάδος FALSE: κανονικός υπολογισμός

Έλεγχος πληρότητας

Μετά το IF ρώτα: «Υπάρχει κάποια είσοδος που δεν ξέρω τι θα κάνει;»

1. Μπερδεύω = με ==

```
! Σωστό:  
if (x == 5) then  
    ...  
end if
```

2. Ξεχνάω END IF — κάθε δομή πρέπει να κλείνει σωστά.

3. Γράφω λάθος λογική για διάστημα: για $0 \leq x \leq 10$ θέλουμε $x \geq 0$.and. $x \leq 10$.

4. Λάθος σειρά ELSE IF — μια γενική συνθήκη μπορεί να προλάβει μια ειδικότερη.

5. Ξεχνάω περίπτωση — στις ασκήσεις διερεύνησης γράψε πρώτα όλες τις περιπτώσεις σε χαρτί.

6. Συγκρίνω υπολογισμένα REAL με == χωρίς σκέψη — όταν χρειάζεται, χρησιμοποίησε ανοχή.

□ Κανόνες εξέτασης

Αν η εκφώνηση έχει «αν», υπογράμμισε τη συνθήκη. Αν έχει «διαφορετικά», ψάξε για ELSE. Αν έχει «ανάλογα με», φτιάξε πρώτα λίστα περιπτώσεων.

2.13 Ασκήσεις εφαρμογής

Οι παρακάτω ασκήσεις βασίζονται στην ενότητα «Εντολές Ελέγχου Ροής» των Ασκήσεων 01 και είναι προσαρμοσμένες στη σειρά μελέτης του κεφαλαίου.

1 — Άρτιος ή περιττός □

Διάβασε έναν ακέραιο και έλεγξε αν είναι άρτιος. Χρησιμοποίησε $\text{MOD}(n,2)$.

2 — Έλεγχος ακτίνας □

Διάβασε ακτίνα σφαίρας. Για αρνητική ακτίνα εμφάνισε μήνυμα. Διαφορετικά υπολόγισε όγκο και εμβαδόν.

3 — Πρωτοβάθμια εξίσωση □□

Για $ax=b$, διάβασε a, b και διερεύνησε: $a \neq 0$, $a=0$ και $b=0$, $a=0$ και $b \neq 0$.

4 — Κατηγοριοποίηση □□

Διάβασε βαθμό 0-10. Για μη έγκυρη είσοδο εμφάνισε μήνυμα και για έγκυρη είσοδο χρησιμοποίησε ELSE IF για κατηγορίες.

5 — Έλεγχος διαστήματος □□

Διάβασε θερμοκρασία και έλεγξε αν βρίσκεται στο $[22, 25.5]$.

6 — Εμφωλευμένο IF □□□

Διάβασε a, b, c , υπολόγισε $d=b^2-4ac$ και χρησιμοποίησε εμφωλευμένο IF ώστε πρώτα να ελέγξεις αν $a=0$ και μετά το πρόσημο της d .

Πώς να τις λύσεις

Για κάθε άσκηση: Δεδομένα → Περιπτώσεις → Συνθήκες → Κλάδοι → Κώδικας. Οι λύσεις βρίσκονται σε ξεχωριστό PDF.

2.14 Checklist πριν προχωρήσουμε

Είσαι έτοιμος να προχωρήσεις στις επαναλήψεις όταν μπορείς, χωρίς αντιγραφή, να κάνεις τα παρακάτω:

- ☐ Να εξηγείς ότι μια λογική έκφραση αξιολογείται σε TRUE/FALSE.
- ☐ Να χρησιμοποιείς `==`, `/=`, `>`, `<`, `>=`, `<=`.
- ☐ Να χρησιμοποιείς `.AND.`, `.OR.`, `.NOT`.
- ☐ Να γράφεις σωστά `IF ... THEN ... END IF`.
- ☐ Να γράφεις `IF ... THEN ... ELSE ... END IF`.
- ☐ Να οργανώνεις περισσότερες περιπτώσεις με `ELSE IF`.
- ☐ Να αναγνωρίζεις πότε χρειάζεται εμφωλευμένο `IF`.
- ☐ Να ελέγχεις εγκυρότητα εισόδου πριν από υπολογισμούς.
- ☐ Να μετατρέπεις «αν / διαφορετικά / ανάλογα» σε συνθήκες.
- ☐ Να αναγνωρίζεις πότε χρειάζεται ανοχή στη σύγκριση `REAL`.

→ Τι έρχεται μετά

Στο επόμενο κεφάλαιο θα βάλουμε τις συνθήκες μέσα σε επαναλήψεις: `DO`, `DO WHILE`, `EXIT`, `CYCLE` και εμφωλευμένους βρόχους. Το Lab 5 χρησιμοποιεί ακριβώς αυτές τις δομές.

Πηγές

- `FORTRAN_2025.pdf` — λογικοί/συγκριτικοί τελεστές, `IF`, `ELSE IF`, εμφωλευμένα `IF` και σύγκριση πραγματικών.
- `Ασκήσεις_01.pdf` — Εντολές Ελέγχου Ροής.
- `Lab_5_presentation_2025_2026_students.pdf` — σύνδεση συνθηκών με `DO WHILE`, `EXIT` και `CYCLE`.